

CMP3006 TERM PROJECT REPORT

*A Comparative Study of Ant Colony Optimization and
Simulated Annealing for the Traveling Salesman Problem*

2366778 Çaęla Özenç
2200923 Efekan Aşkın
2366337 Gözde Korkut
2101434 Natasha Babapour Lashanlou

Abstract

The Traveling Salesman Problem (TSP) is a classical NP-hard optimization problem with important applications in logistics and routing. In this project, we investigated the effectiveness of two metaheuristic approaches, Ant Colony Optimization and Simulated Annealing, for solving the metric TSP. We formally defined the problem, discussed its computational complexity, and motivated the use of heuristic methods for practical problem sizes.

We implemented both algorithms and evaluated their performance on problem instances of varying size. Our experimental results showed that both methods consistently produced optimal solutions for small instances. For larger instances, Simulated Annealing achieved significantly lower runtimes while maintaining competitive solution quality, whereas Ant Colony Optimization required more computation time due to its population-based design.

These findings demonstrate the practical trade-offs between exploration, solution quality, and scalability in heuristic optimization methods.

1. Introduction & Problem Modeling

1.1 Real-World Motivation

Routing and scheduling problems are fundamental challenges in real-world logistics and transportation systems. Industries such as medical supply distribution or courier services rely on efficient route planning to reduce delivery times, operational costs, and resource consumption. In medical logistics in particular, delays or inefficiencies can directly impact service quality and reliability, making route optimization a critical operational task.

The scenario considered in this project is motivated by such real-world applications and is framed using a fictional logistics company. While the company itself is hypothetical, the underlying problem is representative of practical routing challenges faced by logistics providers operating under time constraints and cost limitations. As the number of delivery locations increases, computing optimal routes becomes infeasible in practice due to the rapid growth of the solution space.

Consequently, real-world logistics systems typically favor algorithms that can produce *high-quality solutions within limited time*, rather than exact solutions that are computationally expensive. This trade-off motivates the use of heuristic and metaheuristic approaches, which aim to balance solution quality with scalability and computational efficiency.

1.2 From Story to Graph Model

The routing problem introduced above can be translated into a graph-based model. Each delivery location is represented as a vertex, and traveling between two locations is represented as an edge connecting the corresponding vertices. Since it is assumed that a delivery vehicle can travel directly between any pair of locations, the graph is complete and undirected. The depot, which serves as both the starting and ending point of the route, is included as one of the vertices.

To model travel costs, each location is assigned a pair of coordinates in a two-dimensional plane. The cost of traveling between two locations is defined as the Euclidean distance between their coordinates. This distance measure closely reflects real-world geographic distance and provides a way to quantify route length.

An important property of Euclidean distance is that it satisfies the triangle inequality, ensuring that direct travel between two locations is never longer than an indirect route. Because of this property, our problem belongs to the class of metric Traveling Salesman Problems (TSP), where the objective is to find a closed tour that visits each location exactly once while minimizing total travel distance.

1.3 Formal Definition of the Optimization Problem

Let $G = (V, E)$ be a complete undirected graph where each vertex $v \in V$ represents a delivery location and $|V| = n$. Let $d(u, v)$ denote the distance between vertices u and v , defined using Euclidean distance. A tour is defined as a Hamiltonian cycle that visits each vertex exactly once and returns to the starting vertex.

For a given tour $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, the total tour length is defined as

$$L(\pi) = \sum_{i=1}^n d(\pi_i, \pi_{i+1}),$$

where $\pi_{n+1} = \pi_1$.

The optimization version of the Traveling Salesman Problem aims to find a tour π^* that minimizes $L(\pi)$, subject to the constraint that each vertex is visited exactly once. In other words, the objective is to determine the shortest possible closed route that visits all locations and returns to the starting point.

1.4 Formal Definition of the Decision Problem

In addition to the optimization formulation, we consider the decision version of the Traveling Salesman Problem for the purpose of computational complexity analysis. The input to the decision problem consists of a complete undirected graph $G = (V, E)$ a distance function d that satisfies the triangle inequality, and a positive real number z representing a distance bound.

The decision question is defined as follows: Does there exist a tour that visits each vertex exactly once, returns to the starting vertex, and has total length at most z ? If such a tour exists, the instance is classified as a **YES** instance; otherwise, it is a **NO** instance.

This formulation does not aim to compute the shortest possible tour. Instead, it provides a yes/no version of the problem that is suitable for analyzing computational complexity and establishing NP-hardness.

1.5 Linking Optimization and Decision Versions

The optimization and decision formulations describe the same underlying routing problem but serve different purposes. The optimization version focuses on finding the shortest possible tour, which is the objective addressed by algorithms in real-world applications. In contrast, the decision version transforms the problem into a yes/no question by introducing a distance bound.

In this project, we applied heuristic algorithms directly to the optimization problem in order to compute high-quality tours efficiently. We used the decision formulation only for

theoretical purposes, such as analyzing computational complexity and NP-hardness. This separation between theory and practice explains why we focus on complexity analysis in the next section.

2. Complexity & NP-Hardness Analysis

2.1 Why Complexity Matters

As the number of delivery locations increases, the number of possible tours grows extremely fast. For a problem with n locations, there are $(n - 1)!$ possible tours, which makes it computationally infeasible to examine all possibilities even for small scaled instances. This explains why exact algorithms that guarantee optimal solutions do not scale well in practice.

In real-world routing applications, such as logistics and transportation, decisions must often be made within strict time limits. As a result, algorithms that require exponential time quickly become impractical as the problem size increases. This limitation motivates the use of heuristic and metaheuristic approaches, which aim to find good solutions within reasonable computation time rather than guaranteeing optimality.

In this project, we focused on understanding the computational difficulty of the Traveling Salesman Problem and how this difficulty influences algorithm design. Recognizing the problem's inherent complexity justified our decision to use approximation-oriented methods instead of exact optimization techniques.

2.2 NP Membership

To analyze the computational complexity of the decision version of the Traveling Salesman Problem, we first considered whether it belongs to the class NP. A problem is in NP if a proposed solution can be verified in polynomial time.

For the decision version of the Traveling Salesman Problem, a natural certificate is simply a proposed tour that specifies the order in which all vertices are visited. Given such a tour, we can verify it efficiently by checking that every vertex appears exactly once and that the tour returns to the starting point. We can then compute the total length of the tour by adding the distances between consecutive vertices and compare the result with the given bound z .

Since all these verification steps can be completed in polynomial time, we concluded that the decision version of the Traveling Salesman Problem belongs to the class NP.

2.3 NP-Hardness via Reduction

To show that the decision version of the Traveling Salesman Problem is NP-hard, we reduced a known NP-complete problem to it. We chose the Hamiltonian Cycle problem, since it is closely related to TSP and allows for a clear and intuitive reduction.

In the Hamiltonian Cycle problem, we are given a graph and asked whether there exists a cycle that visits each vertex exactly once and returns to the starting vertex. Given an instance of this problem, we constructed a corresponding TSP instance by creating a complete graph on the same set of vertices. Edges that exist in the original graph were assigned a distance of 1, while edges that do not exist were assigned a distance of 2.

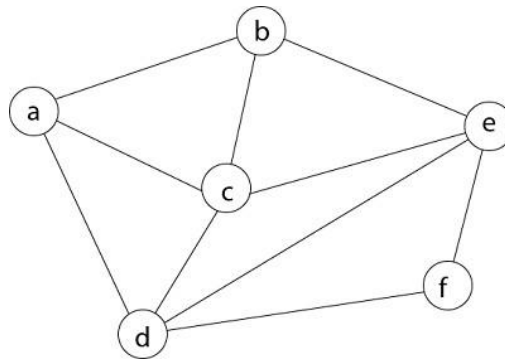


figure 1: Visualization of the Hamiltonian Cycle Problem

We then set the distance bound z to be equal to the number of vertices in the graph. If the original graph contains a Hamiltonian cycle, then there exists a tour in the constructed TSP instance that uses only edges of weight 1 and has total length at most z . Conversely, if a tour of total length at most z exists in the TSP instance, it must correspond to a Hamiltonian cycle in the original graph.

Since this transformation can be performed in polynomial time and a solution to the TSP instance directly implies a solution to the Hamiltonian Cycle instance, we concluded that the decision version of the Traveling Salesman Problem is NP-hard.

2.4 Approximation Perspective

Since the Traveling Salesman Problem is NP-hard, it is natural to consider approximation approaches. For the metric version of TSP, where the triangle inequality holds, approximation algorithms with proven guarantees exist, however, these guarantees do not extend to the general version of TSP.

In practice, even for metric TSP, approximation algorithms may still be too restrictive or complex for large-scale applications. For this reason, we focused on heuristic and metaheuristic methods, which do not offer formal approximation guarantees but often provide high-quality solutions efficiently in practice.

3. Data & Instance Generation

3.1 Instance Sizes

To evaluate the performance of the proposed algorithms under different problem scales, we generated three synthetic Traveling Salesman Problem instances of varying sizes: small, medium, and large.

The small instance contained 12 cities, which allowed us to compute the exact optimal solution for comparison. The medium and large instances contained 50 and 100 cities, respectively, and were used to study scalability and runtime behavior.

Using multiple instance sizes enabled us to analyze how solution quality and computation time changed as the number of cities increased.

3.2 Coordinate Generation and Distance Computation

For each instance, city locations were generated as points in a two-dimensional plane. The coordinates were sampled randomly within a fixed square region, ensuring that all instances were generated using the same spatial scale providing a simple and commonly used model for geographic routing problems.

The distance between any two cities was computed using Euclidean distance which we have covered in previous sections. This distance metric reflects real-world geographic travel while also satisfying the triangle inequality, which is required for the problem to be classified as a metric Traveling Salesman Problem.

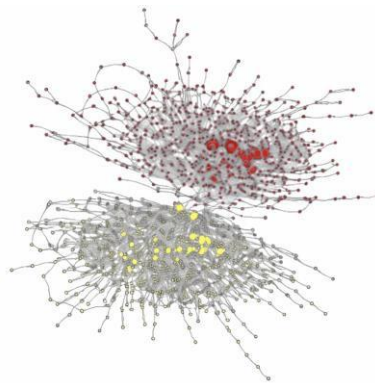


figure 2: Network of a Randomly Generated TSP Instance

3.3 Exact Solutions for Small Instances

For the small instance containing 12 cities, we computed the exact optimal tour using an exhaustive search approach. Because the number of possible tours grows factorially with the number of cities, this approach is only feasible for very small instances. Nevertheless, obtaining the exact solution for the small case allowed us to evaluate the approximation quality of our heuristic algorithms.

For medium and large instances, computing exact solutions was not practical due to the exponential growth of the search space. Therefore, these instances were used solely to compare the relative performance of Ant Colony Optimization and Simulated Annealing.

4. Methodology: Algorithms

4.1 Ant Colony Optimization (ACO)

We implemented ACO as a metaheuristic approach for solving the Traveling Salesman Problem. ACO is inspired by the foraging behavior of real ants, which communicate indirectly by depositing pheromones on paths. Over time, shorter paths accumulate more pheromone and become more attractive, guiding future ants toward better solutions.

In our implementation, each ant constructed a complete tour by incrementally selecting the next city to visit. The probability of choosing a particular city depended on two factors: the pheromone level on the corresponding edge and a heuristic value, defined as the inverse of the distance between cities. These two components were weighted using parameters α and β , which controlled the relative importance of pheromone information and heuristic guidance.

Initially, pheromone values on all edges were set uniformly. After all ants completed their tours in an iteration, pheromone values were updated in two steps: First, pheromone evaporation was applied to reduce all pheromone levels by a fixed evaporation rate ρ , preventing premature convergence. Then, additional pheromone was deposited on edges that were part of shorter tours, reinforcing higher-quality solutions.

The algorithm was repeated for a fixed number of iterations, and the shortest tour found across all iterations was recorded as the final solution produced by ACO.

Below lies the pseudocode for the ACO algorithm.

PROCEDURE InitializePheromones(n , τ_0) $\rightarrow$ $\tau[0..n-1][0..n-1]$

1. $\tau \leftarrow n \times n$ matrix filled with τ_0
2. for $i \leftarrow 0$ to $n-1$ do $\tau[i][i] \leftarrow 0$
3. return τ

PROCEDURE ChooseNextCity($current$, $visited[0..n-1]$, D , τ , α , β , RNG) $\rightarrow$ int

// This matches main.cpp exactly: weights are NOT normalized.

1. $n \leftarrow \text{size}(visited)$
2. $weight[0..n-1] \leftarrow$ all zeros
3. $sum \leftarrow 0$
4. for $j \leftarrow 0$ to $n-1$ do
5. if $visited[j] = \text{false}$ then
6. $dist \leftarrow D[current][j]$
7. if $dist > 0$ then $\eta \leftarrow 1/dist$ else $\eta \leftarrow 1e9$
8. $w \leftarrow (\tau[current][j]^\alpha) * (\eta^\beta)$
9. if w is not finite OR $w < 0$ then $w \leftarrow 0$
10. $weight[j] \leftarrow w$
11. $sum \leftarrow sum + w$
12. if $sum \leq 0$ then
13. // fallback in the code: return the FIRST unvisited city
14. for $j \leftarrow 0$ to $n-1$ do
15. if $visited[j] = \text{false}$ then return j
16. return -1


```

17. r <- UniformRandom(0, sum) using RNG
18. cumulative <- 0
19. for j<-0 to n-1 do
20.     if visited[j] = false then
21.         cumulative <- cumulative + weight[j]
22.         if cumulative >= r then return j
23. // safety fallback (same as code)
24. for j<-0 to n-1 do
25.     if visited[j] = false then return j
26. return -1

```

PROCEDURE BuildAntTour(startCity, D, tau, alpha, beta, RNG) -> tour[0..n-1]

```

1. n<- number of cities
2. visited[0..n-1] <- all false
3. tour <- empty list (capacity n)
4. current <- startCity
5. visited[current] <- true
6. append current to tour
7. while size(tour) < n do
8.     next <- ChooseNextCity(current, visited, D, tau, alpha, beta, RNG)
9.     if next == -1 then break
10.    visited[next] <- true
11.    append next to tour
12.    current <- next
13. if size(tour) < n then
14.     remaining <- all cities j with visited[j] = false
15.     Shuffle(remaining, RNG)
16.     append all remaining to tour
17. return tour

```

PROCEDURE Evaporate(tau, rho)

```

1. factor <- 1 - rho
2. minTau <- 1e-12
3. for i<-0 to n-1 do
4.     for j<-0 to n-1 do
5.         tau[i][j] <- tau[i][j] * factor
6.         if tau[i][j] < minTau then tau[i][j] <- minTau
7.     tau[i][i] <- 0

```

PROCEDURE Deposit(tau, tour, L, Q)

```

1. if L <= 0 OR L is not finite: return
2. add <- Q/L
3. for i<-0 to n-2 do
4.     a <- tour[i]; b <- tour[i+1]
5.     tau[a][b] <- tau[a][b] + add
6.     tau[b][a] <- tau[b][a] + add
7. last <- tour[n-1]; first <- tour[0]
8. tau[last][first] <- tau[last][first] + add
9. tau[first][last] <- tau[first][last] + add
10. for i<-0 to n-1 do tau[i][i] <- 0

```

PROCEDURE RunACO(D, iterations, numAnts, tau0, alpha, beta, rho, Q, RNG)
 -> (bestLen, bestTour)

1. $n \leftarrow \text{size}(D)$
2. $\tau \leftarrow \text{InitializePheromones}(n, \tau_0)$
3. $\text{globalBestLen} \leftarrow +\infty$
4. $\text{globalBestTour} \leftarrow \text{empty}$
5. for $it \leftarrow 1$ to iterations do
 6. $\text{bestLenIter} \leftarrow +\infty$
 7. $\text{bestTourIter} \leftarrow \text{empty}$
 8. repeat numAnts times:
 9. $\text{startCity} \leftarrow \text{RandomInteger}(0, n-1)$
 10. $\text{tour} \leftarrow \text{BuildAntTour}(\text{startCity}, D, \tau, \alpha, \beta, \text{RNG})$
 11. if $\text{IsValidTour}(\text{tour}, n) = \text{false}$: continue
 12. $\text{len} \leftarrow \text{TourLength}(\text{tour}, D)$
 13. if $\text{len} < \text{bestLenIter}$ then
 14. $\text{bestLenIter} \leftarrow \text{len}$
 15. $\text{bestTourIter} \leftarrow \text{tour}$
 16. if bestTourIter not empty AND $\text{bestLenIter} < \text{globalBestLen}$ then
 17. $\text{globalBestLen} \leftarrow \text{bestLenIter}$
 18. $\text{globalBestTour} \leftarrow \text{bestTourIter}$
 19. $\text{Evaporate}(\tau, \rho)$
 20. // IMPORTANT: matches main.cpp -> deposit ONLY using iteration-best tour
 21. if bestTourIter not empty AND bestLenIter is finite then
 22. $\text{Deposit}(\tau, \text{bestTourIter}, \text{bestLenIter}, Q)$
23. return ($\text{globalBestLen}, \text{globalBestTour}$)

Algorithm 1: Ant Colony Optimization for the Traveling Salesman Problem

4.2 Simulated Annealing

We implemented SA as a second metaheuristic approach for solving the Traveling Salesman Problem. SA is inspired by the physical process of annealing in metallurgy, where a material is gradually cooled in order to reach a low-energy and stable configuration. In the context of optimization, this idea is used to avoid getting trapped in poor local minima by occasionally accepting worse solutions early in the search.

We started the algorithm from an initial feasible tour, which was generated randomly. At each step, a neighboring solution was created by applying a small modification to the current tour, such as swapping the order of two cities. The quality of the new tour was evaluated based on its total length.

If the new tour was shorter than the current one, it was (almost) always accepted. Otherwise, it was accepted with a probability that depended on both the increase in tour length and the current temperature. The temperature was gradually decreased according to a predefined cooling schedule, reducing the likelihood of accepting worse solutions over time.

The algorithm terminated after a fixed number of iterations, and the best tour encountered during the search was returned as the final solution.

Below lies a pseudocode for the algorithm.

PROCEDURE CreateInitialTour(n) -> tour[0..n-1]

1. tour[i] <- i for i = 0..n-1
2. return tour

PROCEDURE ApplyTwoOpt(tour, i, j)

1. reverse the subarray tour[i..j] in place

PROCEDURE RunSA(D, iterations, T0, Tmin, coolingRate, RNG) -> (bestLen, bestTour)

1. n <- size(D)
2. current <- CreateInitialTour(n)
3. Shuffle(current, RNG)
4. currentLen <- TourLength(current, D)
5. best <- current
6. bestLen <- currentLen
7. T <- T0
8. for it <- 0 to iterations-1 while T > Tmin do
 9. i <- RandomInteger(0, n-1)
 10. j <- RandomInteger(0, n-1)
 11. if i > j then swap(i, j)
 12. if i = j then
 13. T <- T * coolingRate
 14. continue
 15. candidate <- copy of current
 16. ApplyTwoOpt(candidate, i, j)
 17. candLen <- TourLength(candidate, D)
 18. delta <- candLen - currentLen
 19. if delta <= 0 then
 20. accept <- true
 21. else
 22. u <- UniformRandom(0, 1)
 23. accept <- (u < exp(-delta / T))
 24. if accept then
 25. current <- candidate
 26. currentLen <- candLen
 27. if currentLen < bestLen then
 28. bestLen <- currentLen
 29. best <- current
 30. T <- T * coolingRate
31. return (bestLen, best)

Algorithm 2: Simulated Annealing for the Traveling Salesman Problem

5. Experimental Setup & Results

5.1 Experimental Setup

We evaluated the performance of Ant Colony Optimization and Simulated Annealing on the generated Traveling Salesman Problem instances described in Section 3. All experiments were conducted on the same computing environment to ensure fair comparison between the algorithms.

For each instance size, we executed multiple independent runs of both algorithms in order to account for randomness in their behavior. During each run, we recorded the total tour length produced by the algorithm as well as the computation time required to obtain the solution. For small instances, where the optimal solution was known, we also computed the approximation ratio by comparing the heuristic solution to the optimal tour length.

Both algorithms were executed with fixed parameter settings across all runs for a given instance size. This allowed us to analyze how each method scaled as the problem size increased, without the results being influenced by per-instance parameter tuning.

5.2 Parameter Settings

To ensure a fair comparison between Ant Colony Optimization and Simulated Annealing, we used fixed parameter settings for each algorithm across all experiments. Parameters were selected to provide stable performance without aggressive tuning for individual instances.

For Ant Colony Optimization, we fixed the number of ants, pheromone evaporation rate, and the relative weights of pheromone and heuristic information throughout all experiments. These parameters were chosen to balance exploration and convergence stability without aggressive tuning.

For Simulated Annealing, we used a fixed initial temperature and a simple cooling schedule that gradually reduced the temperature over time. The same neighborhood operator was applied consistently across all runs. Keeping parameters constant ensured that performance differences reflected algorithmic behavior rather than parameter adjustments.

5.3 Results and Performance Comparison

We compared Ant Colony Optimization and Simulated Annealing in terms of solution quality and runtime across small, medium, and large problem instances. Each algorithm was executed multiple times per instance size, and the reported results represent consistent behavior observed across runs.

For the small instance (12 cities), where the optimal tour length was known, both algorithms consistently found the optimal solution in all runs. This resulted in approximation ratios equal to one for both methods, confirming that ACO and SA were able to solve small instances reliably. However, even in this case, Simulated Annealing required noticeably less computation time than Ant Colony Optimization.

For the medium instance (50 cities), optimal solutions were no longer available. In this setting, Simulated Annealing generally produced slightly shorter tours on average while maintaining significantly lower runtimes. Ant Colony Optimization showed higher computational cost due to its population-based nature, although its solution quality remained competitive across runs.

For the large instance (100 cities), performance differences became more pronounced. Simulated Annealing consistently achieved shorter runtimes, often by an order of magnitude compared to Ant Colony Optimization. While ACO occasionally produced competitive tours, its runtime increased substantially with problem size. This highlights the scalability advantage of Simulated Annealing for larger instances.

Overall, these results demonstrate a clear trade-off between solution quality and computational efficiency. Simulated Annealing favored faster convergence and scalability, whereas Ant Colony Optimization incurred higher runtime overhead in exchange for broader exploration of the solution space.

Instance Size (n)	Avg. ACO Tour Length	Avg. SA Tour Length	Avg. ACO Runtime (ms)	Avg. SA Runtime (ms)
12	326.746	326.746	26.8	13.5
50	627	588	326	26.7
100	916	877	1215	44

Table 1: Average tour length and runtime for ACO and SA across different instance sizes

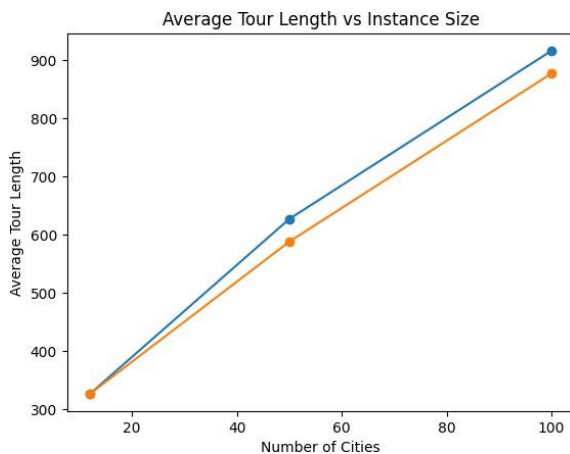


figure 3: Graph of Tour Length vs Instance Size

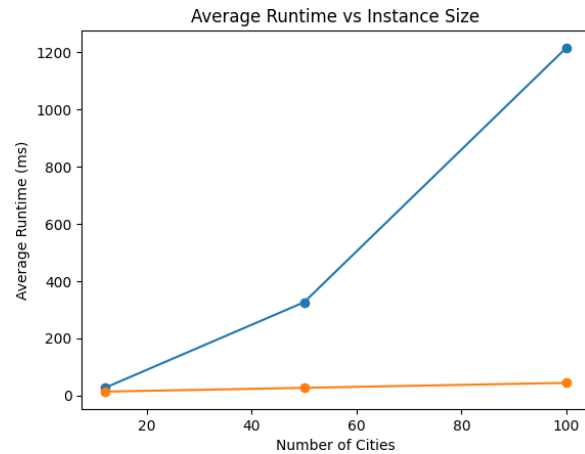


figure 4: Graph of Runtime vs Instance Size

6. Discussion & Reflection

6.1 Comparative Analysis of ACO and SA

Based on the experimental results, both Ant Colony Optimization and Simulated Annealing were able to produce high-quality solutions for the Traveling Salesman Problem, but their behavior differed significantly as the problem size increased. For small instances, both algorithms consistently found the optimal solution, demonstrating that they are reliable when the search space is limited.

As instance size increased, clear differences emerged. Simulated Annealing consistently achieved much lower runtimes, as shown in Figure 4, making it more suitable for larger problem instances where computational efficiency is critical. Ant Colony Optimization, on the other hand, required substantially more computation time due to its population-based structure and pheromone update mechanism.

In terms of solution quality, Ant Colony Optimization occasionally produced slightly competitive tours, but this improvement came at the cost of significantly longer runtimes. Simulated Annealing offered a more favorable balance between solution quality and efficiency, especially for medium and large instances. These observations highlight the trade-off between exploration and computational overhead inherent in the two approaches.

6.2 Approximation Perspective

Although neither Ant Colony Optimization nor Simulated Annealing provides formal approximation guarantees, our experiments showed that both methods produced solutions close to optimal for small instances and maintained reasonable solution quality as problem size increased.

For the 12-city instance, both algorithms consistently achieved the optimal tour, confirming their effectiveness in simple settings.

For larger instances, where optimal solutions were unknown, the algorithms demonstrated stable performance across multiple runs. While slight differences in tour length were observed, particularly in favor of Simulated Annealing, these differences remained small relative to the overall tour length. This suggests that heuristic approaches can offer strong practical approximations even in the absence of theoretical bounds.

From an empirical perspective, Simulated Annealing exhibited more predictable behavior and faster convergence, whereas Ant Colony Optimization emphasized broader exploration. These characteristics align with the expected behavior of the two methods and help explain the observed performance trends.

6.3 Our Insights

Through this project, we gained practical insight into the strengths and limitations of metaheuristic algorithms for combinatorial optimization. While both algorithms are conceptually simple, their performance depends heavily on how they scale with problem size and how they balance exploration and exploitation.

One notable observation was that simpler algorithms, such as Simulated Annealing, can outperform more complex approaches in practice, particularly when runtime constraints are important. Although Ant Colony Optimization offers a rich and biologically inspired framework, its computational overhead became a limiting factor for larger instances.

This project also highlighted the importance of experimental evaluation when working with heuristic methods. Without theoretical guarantees, empirical results play a crucial role in understanding algorithm behavior and guiding practical algorithm selection. Given more time, we would explore adaptive parameter tuning and hybrid approaches that combine the strengths of both methods.

7. Conclusion

In this project, we studied the Traveling Salesman Problem as a representative example of a computationally hard optimization problem and evaluated two metaheuristic approaches: Ant Colony Optimization and Simulated Annealing. By formulating the problem formally and analyzing its computational complexity, we motivated the use of heuristic methods for practical problem sizes.

Through experimental evaluation on instances of increasing size, we observed that both algorithms were capable of producing high-quality solutions. For small instances, both methods consistently found the optimal tour, while for larger instances, differences in scalability became evident.

Simulated Annealing demonstrated significantly lower runtimes and more predictable performance as the number of cities increased, whereas Ant Colony Optimization incurred higher computational costs due to its population-based nature.

Overall, our results highlight the trade-off between solution quality and computational efficiency that is inherent in heuristic optimization. This study confirms that simpler heuristics can be highly effective in practice and that experimental evaluation plays a crucial role when theoretical guarantees are unavailable. Future work could explore adaptive parameter tuning or hybrid approaches that combine the strengths of both algorithms.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Dorigo, M., & Stützle, T. (2004). *Ant colony optimization*. MIT Press.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
<https://doi.org/10.1126/science.220.4598.671>

Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study. In E. H. L. Aarts & J. K. Lenstra (Eds.), *Local search in combinatorial optimization* (pp. 215–310). Wiley.

Dorigo, M. (2007). Ant colony optimization. *Scholarpedia*, 2(3), 1461.
http://www.scholarpedia.org/article/Ant_colony_optimization

Kirkpatrick, S. (n.d.). Simulated annealing. *Wikipedia*.
https://en.wikipedia.org/wiki/Simulated_annealing